



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA EN SISTEMAS COMPUTACIONALES
LICENCIATURA EN DESARROLLO Y GESTIÓN DE SOFTWARE
DESARROLLO DE SOFTWARE VII



LABORATORIO NO. 3

CRUD con Laravel

PROFESOR:

IRINA FONG

ESTUDIANTE:

SHIRLEY WEN, 8-957-1526

GRUPO:

1GS133

I SEMESTRE

2026

INTRODUCCIÓN

Laravel es un framework de desarrollo web basado en PHP que sigue el patrón de arquitectura MVC (Modelo-Vista-Controlador). Fue creado con el objetivo de hacer el desarrollo web más sencillo, elegante y expresivo, proporcionando herramientas integradas para tareas comunes como el manejo de rutas, migraciones de base de datos, autenticación y mucho más.

Un CRUD es el conjunto de las cuatro operaciones básicas que se realizan sobre los datos en una aplicación:

- Create (Crear): agregar nuevos registros a la base de datos.
- Read (Leer): consultar y mostrar registros existentes.
- Update (Actualizar): modificar registros existentes.
- Delete (Eliminar): borrar registros de la base de datos.

En este laboratorio se implementó un CRUD completo para una tabla de productos utilizando Laravel 9, WAMP como servidor local y el paquete `ibex/crud-generator` para generar automáticamente el código necesario.

DESARROLLO DEL LABORATORIO

Paso 1: Levantar el servidor y crear el proyecto

Se inició WAMP Server asegurando que el ícono en la barra de tareas estuviera de color verde, indicando que Apache y MySQL estaban corriendo correctamente. Luego se abrió la terminal en Visual Studio Code y se ejecutó el siguiente comando para crear el proyecto:

```
laravel new crudRapido
```

Una vez creado el proyecto, se abrió con Visual Studio Code y se levantó el servidor de desarrollo de Laravel con:

```
php artisan serve
```

Al acceder a <http://127.0.0.1:8000> se verificó que la pantalla de bienvenida de Laravel cargaba correctamente.

Paso 2: Configurar el archivo .env

Se abrió el archivo .env en la raíz del proyecto y se configuraron las credenciales de la base de datos:

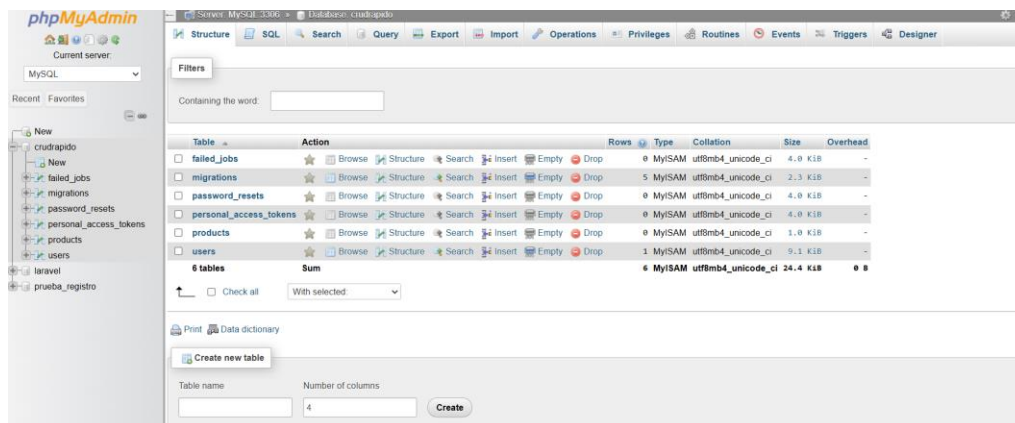
```
DB_DATABASE=crudrapido
```

```
DB_USERNAME=root
```

```
DB_PASSWORD=
```

Se verificó en phpMyAdmin que la base de datos crudrapido existiera. Luego se limpió la caché de configuración con:

```
php artisan config:clear
```



Paso 3: Crear el modelo y la migración

Se ejecutó el siguiente comando para crear el modelo Product junto con su archivo de migración:

```
php artisan make:model Product -m
```

Se abrió el archivo de migración generado en database/migrations/ y se definieron las columnas de la tabla products:

```
$table->id();  
$table->string('description');  
$table->double('price', 8, 2);  
$table->integer('stock');  
$table->timestamps();
```

Paso 4: Ejecutar las migraciones

Se corrió la migración para crear la tabla en la base de datos:

```
php artisan migrate
```

Se verificó en phpMyAdmin que la tabla products fue creada correctamente con todas sus columnas.

Paso 5: Instalar el generador de CRUD

Se instaló el paquete ibex/crud-generator en su versión compatible con PHP 8.0:

```
composer require ibex/crud-generator:^1.4 --dev
```

Luego se publicaron los archivos de configuración del paquete:

```
php artisan vendor:publish --tag=crud
```

Paso 6: Generar el CRUD automáticamente

Se ejecutó el generador indicando el nombre de la tabla (products):

```
php artisan make:crud products
```

Este comando generó automáticamente el controlador ProductController, las vistas (index, create, edit, show) y el layout. Finalmente se actualizó el autoload de Composer:

```
composer dump-autoload
```

Paso 7: Configurar las rutas

Se abrió el archivo routes/web.php y se agregaron las rutas de recursos y autenticación:

```
use App\Http\Controllers\ProductController;
Auth::routes();
Route::resource('products', ProductController::class);
```

Se verificó que todas las rutas estuvieran registradas con:

```
php artisan route:list
```

Paso 8: Configurar el \$fillable en el modelo

Se abrió app/Models/Product.php y se verificó que el array \$fillable tuviera los campos permitidos para asignación masiva:

```
protected $fillable = ['name', 'price', 'stock'];
```

Esto protege la aplicación contra vulnerabilidades de asignación masiva, permitiendo solo los campos definidos.

Paso 9: Instalar dependencias de frontend

Para resolver el error de Vite con los archivos CSS, se instaló laravel/ui y se generó el scaffolding de Bootstrap:

```
composer require laravel/ui
php artisan ui bootstrap --auth
npm install
npm run dev
```

Paso 10: Verificar el CRUD en el navegador

Con el servidor corriendo, se accedió a http://127.0.0.1:8000/products y se verificó el funcionamiento completo del CRUD:

- Se visualizó la lista de productos vacía inicialmente.

- Se creó un nuevo producto completando el formulario (nombre, precio, stock).
- Se verificó que el producto apareciera en la lista.
- Se editó el producto y se confirmaron los cambios.
- Se eliminó el producto y se confirmó que desapareció de la lista.

Vista general:

[Laravel](#)

- [Shirley](#)
[Logout](#)

Product

No	Name	Price	Stock	
1	Prueba1	10.25	15	Show Edit Delete

Creación:

[Laravel](#)

- [Shirley](#)
[Logout](#)

Create Product

Name

Price

Stock

Edición:

[Laravel](#)

- [Shirley](#)
[Logout](#)

Update Product

Name

Price

Stock

CONCLUSIÓN

Durante este laboratorio se logró implementar exitosamente un CRUD completo utilizando el framework Laravel. A través del uso del paquete ibex/crud-generator se comprobó cómo es posible acelerar el proceso de desarrollo generando automáticamente controladores, vistas y rutas funcionales con un solo comando.

Aprendí la importancia de configurar correctamente el archivo .env para establecer la conexión con la base de datos, así como el uso de migraciones para mantener un control ordenado de la estructura de las tablas.

Finalmente, la estructura MVC de Laravel y cómo las rutas tipo resource generan automáticamente todos los endpoints necesarios para las operaciones Create, Read, Update y Delete, lo que facilita enormemente el desarrollo de aplicaciones web profesionales.